BTVN#6 · Challenge B: Context management

	B1 Compact (bắt buộc)	B2 Code Mode (điểm cộng)
Kết quả	**PASS** nén 3 lần, agent vẫn nhớ lỗi / cách đã thử / ràng buộc / bước tiếp	**PASS** trả lời đúng ground truth ở cả 50 và 10 file
Số đo chính	Tổng input tokens **168,844** (compact) so với **544,591** (không compact) = 31%. Đỉnh context 23,289 so với 64,725	50 file: Code Mode **6,564** input tokens, native **1,622,134** (gấp 247 lần). Raw log vào context: 1,495 so với 968,136 ký tự
Cách làm	`CompactionMiddleware` chạy trước mỗi lần gọi model: rút gọn tool output dài, nhờ LLM viết checkpoint 7 trường, thay lịch sử cũ bằng checkpoint + vài message gần nhất	Tool `run_python`: model viết chương trình, harness chạy trong tiến trình riêng, model chỉ nhận phần chương trình `print`; so với native đọc từng file bằng `read_file`

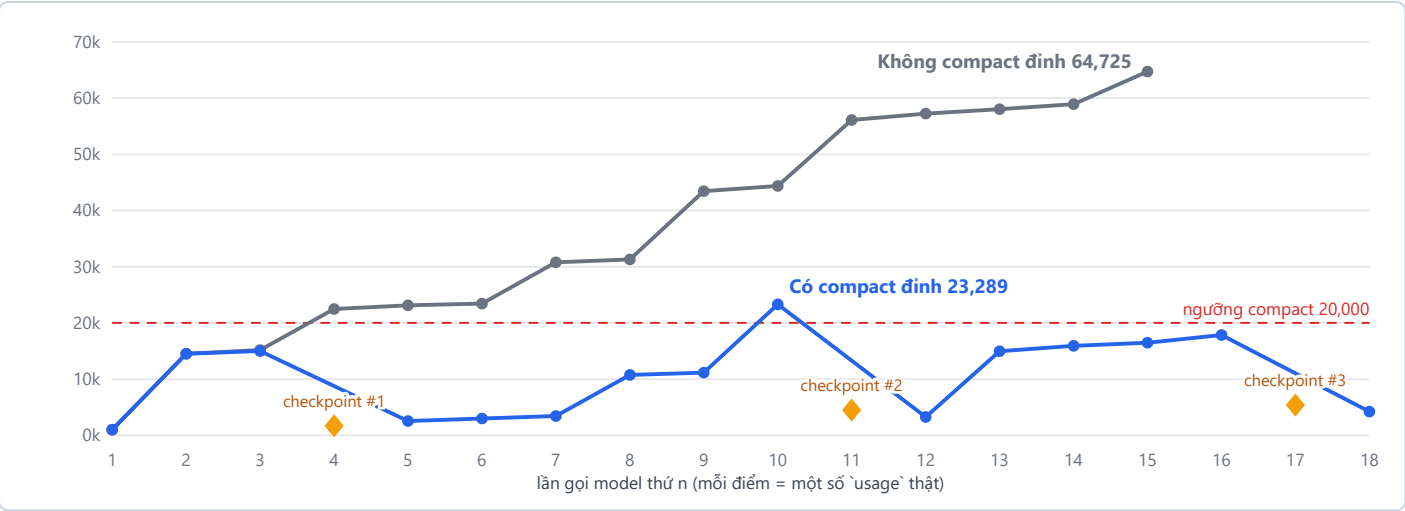
1. B1 Compact: session dài, nén có cấu trúc, agent vẫn làm đúng việc

Bước pipeline (slide tr. 26)	Cài đặt (`compaction.py` , `pruning.py`)	Slide
Observe token budget	usage thật của AIMessage gần nhất + ước lượng phần mới; ngưỡng 20,000 token (cố ý nhỏ để demo)	tr. 25
Prune tool output	giữ dòng ERROR/FAIL (gộp trùng, đếm số lần), gộp dòng lặp, raw lưu `traces/archive/`	tr. 19, 23
Build checkpoint	LLM + structured output, 7 trường `goal … next_action`; trường rỗng thì cảnh báo	tr. 22
Reinject / Resume	system prompt do `create_agent` gắn lại mỗi lần gọi; ràng buộc user nằm trong `constraints`; hỏi lại + cho làm tiếp để kiểm	tr. 22, 26
Chọn điểm cắt	giữ nguyên 4 message cuối, lùi điểm cắt về chỗ không làm ToolMessage mồ côi (nếu không API báo lỗi 400); yêu cầu hiện tại của user được ghi nguyên văn vào checkpoint	tr. 23
Cách kiểm	kịch bản debug giả (3 log ~340-380 dòng + 366 dòng test, sinh cố định): 5 lượt user, rồi hỏi lại và cho làm tiếp; chạy thêm một lần không compact để đối chứng	tr. 32-33

```
$ python run_demo_b1.py (trích demo-b1.txt: phiên dài, tool output lớn, compact kích hoạt)

T1 USER: Mình đang điều tra lỗi: API POST /checkout thỉnh thoảng trả HTTP 500. Ràng buộc: KHÔNG đổi API công khai /checkout
(path, request, response giữ nguyên) và KHÔNG restart production. Bước đầu: đọc checkout-api.log và cho mình biết lỗi chính.
  [tool call] read_log({'name': 'checkout-api.log'})
  [tool result] read_log: 33,602 ký tự | == checkout-api.log (380 dòng) ==
AGENT: [… lược bớt câu trả lời của agent …]
  [ctx] 4 messages; input_tokens lần gọi model cuối của lượt: 14507
T2 USER: Chạy bộ test xem test nào đang fail.
=== COMPACTED: 21,401 → 1,112 tokens (trước: usage; sau: ước lượng ký tự/4, chưa tính system prompt và tool schema) ===
messages 7 → 5; tool output đã archive: ['traces/archive/call_00_ET_NfG2cuWsOR6N3h3xRYjl8351.txt',
'traces/archive/call_00_ET_GpJQW4LzbfUvIMI2dj2J6058.txt']
CẢNH BÁO: checkpoint thiếu trường ['failed_attempts']
```

Hình 1. Tool output lớn (xem cột ký tự); vượt ngưỡng thì compact chạy lần đầu: 21,401 token, 7 → 5 messages.



Hình 2. Input tokens của từng lần gọi model. Không compact (xám): tăng dần theo từng tool output. Có compact (xanh): mỗi lần vượt ngưỡng lại rơi về vài nghìn (hình thoi cam = lần gọi LLM tạo checkpoint).

```
$ python run_demo_b1.py (trích demo-b1.txt: compact lần 2 và checkpoint thật)

=== COMPACTED: 24,360 → 1,607 tokens (trước: usage; sau: ước lượng ký tự/4, chưa tính system prompt và tool schema) ===
messages 17 → 5; tool output đã archive: ['traces/archive/call_00_B1xt6IqdpcyWTNof5XtJ6041.txt',
'traces/archive/call_00_ET_v7u92RoCz0VUPyMLu1mK8455.txt']
[CHECKPOINT] Trạng thái đã nén của phần hội thoại cũ (thay cho các message đã bị bỏ). Coi đây là nguồn sự thật. Các message
đứng ngay sau checkpoint là phần gần nhất, giữ nguyên và MỚI HƠN checkpoint: nếu lệch nhau thì theo các message đó.
goal: Điều tra nguyên nhân lỗi API POST /checkout thỉnh thoảng trả HTTP 500 và đề xuất/áp dụng cách khắc phục.
current_state: Đã đọc checkout-api.log và chạy test 2 lần (trước/sau khi tăng DB pool timeout lên 30s) — kết quả không đổi: 3
failed, 200 passed. Đã xác định nguyên nhân gốc: trong app/services/order_service.py:41, khối `with db.connection() as conn:`
bao trùm cả lời gọi payment gateway nên connection bị giữ suốt thời gian chờ HTTP ra ngoài → pool 20 cạn → 500. Đã báo cáo cho
người dùng và đề xuất 2 lựa chọn (a) revert timeout, (b) revert timeout + fix nhả connection trước khi gọi gateway. Người dùng
chưa chọn; thay vào đó yêu cầu đọc thêm db-pool.log — việc này CHƯA thực hiện. Timeout 30s hiện vẫn đang được áp dụng (chưa
revert).
constraints:
- KHÔNG đổi API công khai /checkout: path, request, response giữ nguyên
- KHÔNG restart production
findings:
- (×14) ERROR POST /checkout 500 DBPoolExhaustedError: connection pool exhausted (active=20/20, waiting=5..19), rải rác 13:59–
14:09, vd request_id=r-75198 lúc 2026-10-26 13:59:35.446
- (×2) ERROR POST /checkout 400 ValidationError: invalid coupon code (request_id=r-21801) — lỗi client, không liên quan 500
- (×159) INFO Retrying connection to db (attempt 4/5)... — dấu hiệu tắc nghẽn kết nối DB kéo dài
- (×15) WARN slow query 991–2052ms SELECT * FROM orders WHERE user_id=? — nghi thiếu index user_id, giữ connection lâu
- (×134) POST /checkout 200 403ms; (×18) GET /health 200 4ms; (×18) cache hit key=cart:1855; dòng cuối log 14:09:14.981 POST
/checkout 200 317ms request_id=r-37535
- run_tests lần 1: 3 failed, 200 passed in 11.82s — test_checkout.py::test_checkout_concurrent_orders (pool exhausted
active=20/20 waiting=14), test_checkout.py::test_checkout_burst_100_requests (waiting=17),
test_payment.py::test_charge_releases_db_connection (AssertionError: expected 0 DB connections held during gateway call, got 1)
- Traceback: tests/test_payment.py:54 client.post("/checkout") → app/api/checkout.py:57 order_service.create(cart, user) →
app/services/order_service.py:41 `with db.connection() as conn:` → app/db/pool.py:112 self._pool.acquire(timeout=self.timeout)
→ app/db/pool.py:78 raise DBPoolExhaustedError
- run_tests lần 2 (sau khi tăng timeout 30s): 3 failed, 200 passed in 11.82s — y hệt lần 1, không cải thiện
failed_attempts:
- Tăng timeout lấy connection từ DB pool lên 30 giây qua apply_fix (app/db/pool.py hoặc DB_POOL_TIMEOUT=30s, không đổi pool
size, không đổi API) rồi run_tests → vẫn 3 failed, 200 passed, cùng thông báo lỗi; timeout chỉ làm chờ lâu hơn chứ không giải
phóng connection, thậm chí có thể làm request treo 30s rồi vẫn 500
files: checkout-api.log, db-pool.log (chưa đọc), app/services/order_service.py, app/api/checkout.py, app/db/pool.py,
tests/test_checkout.py, tests/test_payment.py
next_action: Đọc db-pool.log, tìm các dòng liên quan: cấu hình pool size/timeout hiện tại, số lần acquire/release, connection
bị giữ lâu (leak), thời điểm cạn pool; đối chiếu với giả thuyết connection bị giữ xuyên suốt lời gọi payment gateway trong
order_service.create (dòng 41) và slow query orders.user_id. Sau đó báo cáo phát hiện và chờ người dùng chọn (a) revert timeout
hay (b) revert timeout + fix nhả connection trước khi gọi gateway.
=== hết checkpoint ===
```

Hình 3. Structured checkpoint đủ 7 trường (slide tr. 22), 17 → 5 messages. Hai ràng buộc user nói ở lượt đầu vẫn nằm trong `constraints`.

```
$ python run_demo_b1.py (trích demo-b1.txt: bảng token, số `usage` thật)

BẢNG TOKEN · COMPACT (số liệu `usage` do API trả về)
  # lượt     loại        input_tokens  output cache_hit  ghi chú
  1 T1       agent                998      41       768
  2 T1       agent             14,507     558    14,336
  3 T2       agent             14,991      23    14,464
  4 T2       checkpoint         1,670     679         0  LLM tạo checkpoint (compact #1)
  5 T2       agent              2,523     439       896  ◄ lần gọi đầu SAU compact #1
  6 T3       agent              2,995     303     2,432
  7 T3       agent              3,417      23     3,200
  8 T3       agent             10,739     685     3,328
  9 T4       agent             11,139      53     2,944
 10 T4       agent             23,289   1,061    11,136
 11 T5       checkpoint         4,495   1,227     1,024  LLM tạo checkpoint (compact #2)
 12 T5       agent              3,271      55       896  ◄ lần gọi đầu SAU compact #2
 13 T5       agent             14,974   1,442     3,200
 14 HỎI LẠI  agent             15,911     612     3,200
 15 LÀM TIẾP agent             16,485   1,040    15,872
 16 LÀM TIẾP agent             17,852      23    17,408
 17 LÀM TIẾP checkpoint         5,378   1,688     1,152  LLM tạo checkpoint (compact #3)
 18 LÀM TIẾP agent              4,210     437       896  ◄ lần gọi đầu SAU compact #3
```

Hình 4. Dòng ◄ là lần gọi model đầu tiên sau mỗi lần compact. Lần 2: 23,289 → 3,271 token (giảm 86%).

```
$ python run_demo_b1.py (trích demo-b1.txt: tổng token hai chế độ)

chế độ                    số lần gọi   input tổng  input đỉnh   output  cache hit
COMPACT                           18      168,844      23,289   10,389     97,152
BASELINE (không compact)          15      544,591      64,725    8,148    454,144
Compact tốn 31% input tokens của baseline (đã tính 11,543 input tokens của 3 lần tạo checkpoint). Hai lần chạy do model sinh
nên số lần gọi tool có thể khác nhau chút.
```

Hình 5. Cùng kịch bản, cùng lượt user. Số lần gọi của compact nhiều hơn vì có 3 lần tạo checkpoint, đã tính cả vào tổng.

```
$ python run_demo_b1.py (trích demo-b1.txt: hỏi lại sau compact + tự kiểm)

[COMPACT] câu trả lời sau compact:
**(1) Đang điều tra lỗi gì?**
`POST /checkout` thỉnh thoảng trả HTTP 500. Nguyên nhân: `DBPoolExhaustedError` — pool DB (20 connection, `overflow 0`) bị cạn.
Gốc rễ 2 tầng: payment gateway thỉnh thoảng chậm 8–12s / timeout 12s, và `order_service.py:41` giữ DB connection suốt lời gọi
gateway → vài chục request đồng thời là cạn pool.

**(2) Đã thử cách nào, kết quả?**
- Tăng DB pool timeout lên 30s → **không cải thiện**: vẫn 3 failed / 200 passed y hệt (3 test:
`test_checkout_concurrent_orders`, `test_checkout_burst_100_requests`, `test_payment::test_charge_releases_db_connection`).
Timeout chỉ làm chờ lâu hơn, không nhả connection. Hiện fix này **vẫn đang áp dụng, chưa revert**. Hơn nữa log cho thấy timeout
vốn đã là 30s từ trước.
- Đọc 3 log (`checkout-api`, `db-pool`, `payment-gateway`) → khớp nhau: ~50 lần gateway 8–12s ⇔ `connection held by
payment_client.charge for 8.0–12.9s` ⇔ 500 bắt đầu 13:59:35.

**(3) Ràng buộc đã đặt:**
- Không đổi API công khai `/checkout`: path, request, response giữ nguyên.
- Không restart production.

**(4) Bước tiếp theo đề xuất:**
Chờ bạn chọn:
- **(a)** chỉ revert timeout về giá trị gốc, hoặc
- **(b)** revert timeout + sửa `order_service.create` để nhả connection **trước** khi gọi gateway → chạy `run_tests()`.

Kèm khuyến nghị riêng (ngoài phạm vi test): đặt timeout client payment 3–5s + idempotency key theo `order_id`, vì log có
timeout 12s sau khi đã charge → nguy cơ charge trùng. Bạn chọn (a) hay (b)?
  lỗi đang điều tra: PASS (nhắc /checkout và (500 hoặc pool))
  cách đã thử + kết quả: PASS (nhắc tăng timeout và việc nó không hiệu quả (cụm phủ định gần chữ timeout))
  ràng buộc (không đổi API): PASS (nhắc giữ nguyên API công khai)
  bước tiếp theo hợp lý: PASS (nhắc giải phóng/nhả kết nối DB)
  agent làm tiếp đúng task (áp fix đúng rồi chạy lại test): PASS | fix đã áp: […]
  checkpoint có trường rỗng? [['failed_attempts']]
KẾT QUẢ B1 (chỉ tính lần chạy có compact): PASS
```

Hình 6. Sau compact, agent trả lời đúng cả 4 ý và làm tiếp đúng việc (áp fix đúng rồi chạy lại test). Code chấm bằng keyword, không dựa vào lời agent tự khai.

Rút gọn tool output và bẫy ToolMessage mồ côi (chạy trực tiếp, không cần API)

```
$ python scripts/show_prune_example.py

read_log('checkout-api.log'): thô 381 dòng / 33,602 ký tự → rút gọn 13 dòng / 937 ký tự (2.8%)
run_tests(): thô 366 dòng / 25,549 ký tự → rút gọn 18 dòng / 1,306 ký tự (5.1%)

--- read_log('checkout-api.log') sau khi rút gọn ---
[ĐÃ RÚT GỌN tool output: 381 dòng / 33,602 ký tự → còn ~9 dòng. Bản đầy đủ: traces/archive/<id>.txt (cần raw thì gọi lại
tool).]
== checkout-api.log (380 dòng) ==
-- Dòng lỗi / FAIL (đã gộp trùng) --
(×14) 2026-10-26 13:59:35.446 ERROR [checkout-api] POST /checkout 500 DBPoolExhaustedError: connection pool exhausted
(active=20/20, waiting=5) request_id=r-75198
(×2) 2026-10-26 14:04:08.114 ERROR [checkout-api] POST /checkout 400 ValidationError: invalid coupon code request_id=r-21801
-- Dòng lặp đã gộp --
(×159) INFO  [checkout-api] Retrying connection to db (attempt 4/5)...
(×134) INFO  [checkout-api] POST /checkout 200 403ms request_id=r-23659
(×38) INFO  [checkout-api] GET /health 200 4ms
(×18) INFO  [checkout-api] cache hit key=cart:1855
[… lược bớt …]
```

Hình 7. Slide tr. 23: giữ lỗi chưa giải quyết, rút gọn output dài, bỏ log trùng (gộp thành ×N). 33,602 ký tự còn 937.

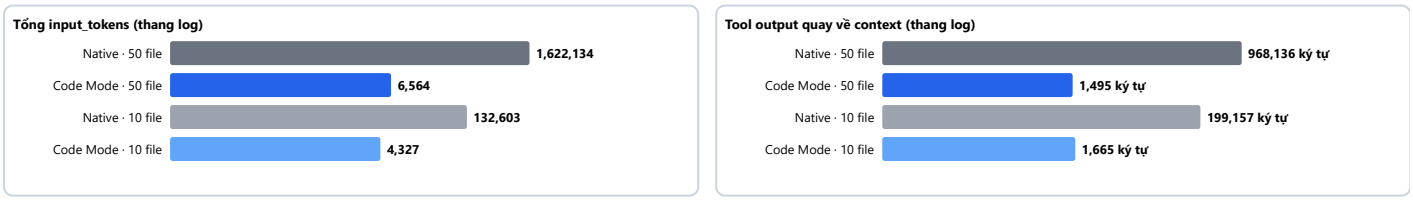
```
$ python test_offline.py

OK find_cutoff: mọi điểm cắt cho đuôi hợp lệ; orphan_tool_messages bắt được cắt sai
OK measure_context_tokens: ưu tiên usage thật, usage đã bỏ thì quay về ước lượng
OK prune: log 33,602 → 934 ký tự; test 25,549 → 1,303 ký tự, đủ 3 FAILED
OK debug_tools: deterministic; fix sai vẫn fail, fix đúng mới pass
OK run_python: không lộ API key, bắt lỗi, timeout, cắt output
OK middleware: giữ nguyên văn yêu cầu hiện tại, đuôi hợp lệ, tool output đuôi được rút gọn
OK middleware: đầu chỉ có checkpoint cũ thì chỉ rút gọn đuôi, không gọi LLM
OK checkpoint: trường thiếu không làm chết phiên, lỗi tạo checkpoint được thử lại
OK grader: câu đúng qua cả 4 ý; câu sai (timeout hiệu quả, tăng pool size) rớt đúng 2 ý
TẤT CẢ OK
```

Hình 8. Bẫy chính của compact: đuôi giữ lại không được bắt đầu bằng ToolMessage mồ côi (API báo 400). Test thử mọi điểm cắt, cả trường hợp gọi tool song song.

2. B2 Code Mode: 50 file log, raw log không vào context

- **Code chạy ở đâu:** LLM chỉ viết chương trình (tool call `run_python`); **harness** chạy nó trong tiến trình Python riêng, LLM chỉ nhận phần chương trình `print` (slide tr. 28).
- **Giới hạn:** cwd = `workspace/`, timeout 15 giây, env không có API key, output cắt ở 2,000 ký tự. **Không phải sandbox bảo mật** (slide tr. 31).
- **So sánh:** cùng một câu hỏi, cùng bộ log, hai chế độ. Native dùng `list_files` + `read_file`; Code Mode chỉ dùng `run_python` (chương trình chỉ in kết quả cuối). Đo tổng `input_tokens` bằng `usage`, chấm đúng/sai với ground truth, chạy ở 50 file và 10 file.
- **Bộ dữ liệu:** 50 file, 11,411 dòng, đáp án đếm bằng cách parse lại chính các file (ground truth ngoài `workspace/`); bẫy: nhiều dòng WARN nhắc đúng tên lỗi, chỉ tính dòng ERROR.



Hình 9. Cùng câu hỏi, cùng bộ log, thang log vì chênh nhau hàng trăm lần (50 file: native gấp 247 lần).

```
$ python run_demo_b2.py (trích demo-b2.txt: bảng so sánh)

SO SÁNH (cùng câu hỏi, cùng bộ log; token đo bằng `usage` của API)
================================================================================
chế độ  file lần gọi  tool  input tổng  input đỉnh  tool output→context  đúng?  trạng thái
native    50       8    51   1,622,134     412,843          968,136 ký tự   ĐÚNG  OK
code      50       4     3       6,564       2,251            1,495 ký tự   ĐÚNG  OK
native    10       4    11     132,603      84,968          199,157 ký tự   ĐÚNG  OK
code      10       3     2       4,327       2,013            1,665 ký tự   ĐÚNG  OK
50 file: native tốn gấp 247.1 lần input tokens của Code Mode.
10 file: native tốn gấp 30.6 lần input tokens của Code Mode.

KẾT QUẢ B2 (Code Mode trả lời đúng ground truth ở mọi cỡ): PASS
```

Hình 10. Cả hai chế độ đều đúng; khác biệt nằm ở token. Native không tràn context vì model này có context rất lớn (đỉnh 412,843 token).

```
$ python run_demo_b2.py (trích demo-b2.txt: Code Mode, 50 file)

USER: Trong các file log trong thư mục logs/ (toàn bộ 50 file service-01.log … service-50.log), chỉ tính các dòng có mức ERROR
(không tính WARN hay INFO): lỗi nào xuất hiện nhiều nhất? Cho biết tên lỗi, tổng số lần xuất hiện trên toàn bộ các file đó, và
file có nhiều dòng ERROR của lỗi này nhất (kèm số lần trong file đó). Tên lỗi là từ đứng ngay sau 'ERROR [service-xx] ' và
trước dấu ':'.
[kết thúc] trạng thái: OK | 4 lần gọi model | 3 lần chạy tool | 6s
Chương trình model viết ở lần chạy cuối:
import os,re,collections
pat=re.compile(r'^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2} ERROR \[service-\d+\] ([A-Za-z0-9_.\-]+):')
per=collections.defaultdict(collections.Counter)
for f in sorted(os.listdir('logs')):
    for ln in open('logs/'+f):
        m=pat.match(ln)
        if m: per[m.group(1)][f]+=1
for n in ['TimeoutError','ConnectionResetError']:
    mx=max(per[n].values())
    print(n,[ (f,c) for f,c in per[n].most_common() if c==mx])
AGENT: **Kết quả**

- Lỗi ERROR xuất hiện nhiều nhất: **TimeoutError**
- Tổng số lần trên toàn bộ 50 file: **174 lần**
- File có nhiều ERROR TimeoutError nhất: **service-37.log – 16 lần**

Bối cảnh: tổng cộng 11.411 dòng log, trong đó **518 dòng ERROR** (chỉ tính mức ERROR), thuộc **6 loại lỗi** khác nhau. Hai loại
tiếp theo là ConnectionResetError (132 lần) và DBPoolExhaustedError (80 lần).
Đáp án đúng: TimeoutError 174 lần; file service-37.log 16 lần
  tên lỗi = TimeoutError: PASS
  tổng = 174: PASS
  file = service-37.log: PASS
  số lần trong file = 16: PASS
```

Hình 11. Model tự viết chương trình đọc 50 file, lọc đúng dòng ERROR bằng regex, chỉ in vài dòng; 4 lần gọi model, 3 lần chạy tool.

```
$ python run_demo_b2.py (trích demo-b2.txt: native, 50 file)

[kết thúc] trạng thái: OK | 8 lần gọi model | 51 lần chạy tool | 51s
AGENT: ## Kết quả

Đã đọc toàn bộ 50 file `logs/service-01.log` … `logs/service-50.log` và chỉ đếm các dòng mức **ERROR** (bỏ qua WARN/INFO).

**Lỗi xuất hiện nhiều nhất: `TimeoutError`**

- **Tổng số lần trên cả 50 file:** **174**
- **File có nhiều ERROR `TimeoutError` nhất:** **`logs/service-37.log` – 16 lần**

[… lược bớt …]
Đáp án đúng: TimeoutError 174 lần; file service-37.log 16 lần
  tên lỗi = TimeoutError: PASS
  tổng = 174: PASS
  file = service-37.log: PASS
  số lần trong file = 16: PASS
```

Hình 12. Native: 8 lần gọi model, 51 lần chạy tool; mỗi lần gọi gửi lại toàn bộ log đã đọc nên tổng input phình lên.